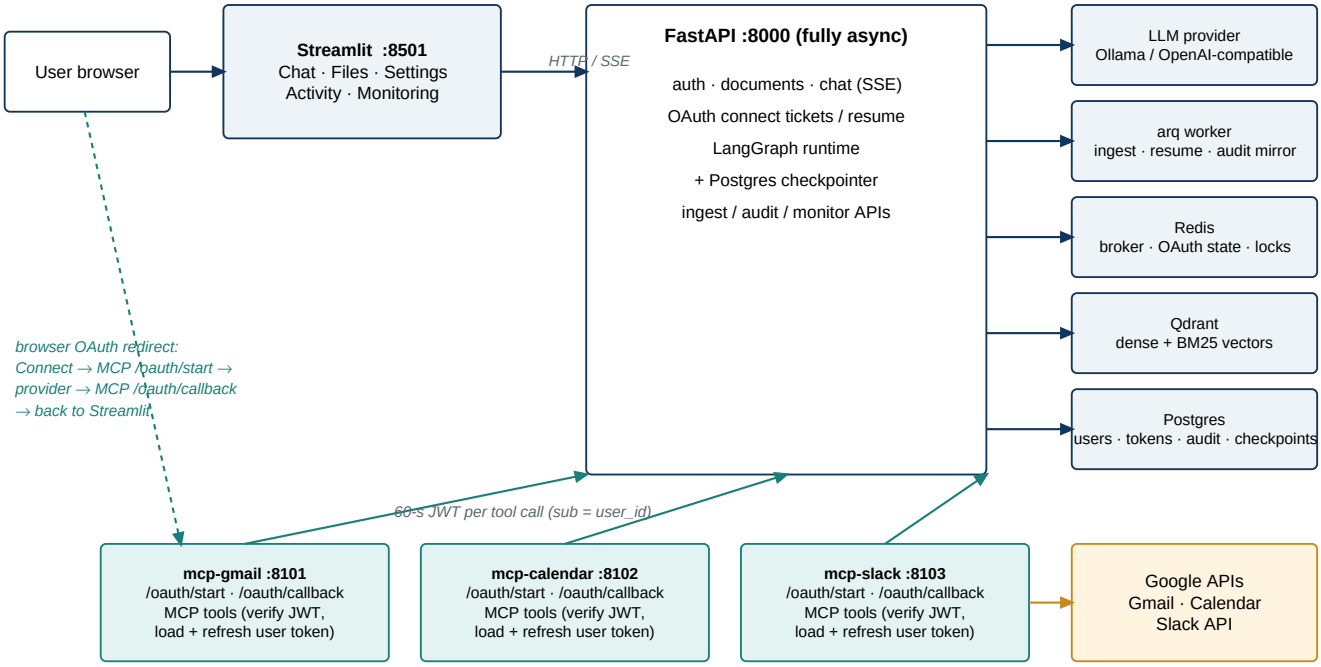
Banking Multi-Agent RAG Chatbot — Design Document

LangGraph · MCP (Gmail, Calendar, Slack) · per-user OAuth2 · Excel/CSV-only RAG · FastAPI · Streamlit | Live demo: (fill in after deployment) | API docs: /docs

1. System Architecture



Request path. The browser talks only to Streamlit. Streamlit holds the user's session JWT in `st.session_state` and calls FastAPI over HTTP; chat uses server-sent events (`step | token | final | interrupt`) so the UI can show which agent is working. FastAPI runs the LangGraph graph in-process with a Postgres checkpointer; heavy or deferred work (embedding, resuming a graph after an officer approves, mirroring audit summaries to Slack) goes through **arq on Redis** to a worker. Agents reach Gmail/Calendar/Slack only through three MCP servers over streamable HTTP; each call carries a 60-second JWT (`sub=user_id, aud=mcp:<service>`) minted by the API from the *graph state's* user id, never from model output. The MCP server, not the API, loads and decrypts that user's token.

Process — File ingestion & validation

- **UI:** `st.file_uploader(type=['xlsx','csv'])` blocks other extensions in the widget. **Server:** `validate_upload` (first step of `save_and_queue`) checks the extension *and* content (xlsx must be a zip with `xl/workbook.xml`; csv must not be binary/zip, so a renamed PDF fails). Rejection returns HTTP 415 and writes a `file_rejected` audit row. Tested in `test_rag_pipeline.py`.
- Accepted bytes are written **encrypted at rest** (Fernet) under a server-generated name, a `documents` row is created, and an arq job is queued; the worker parses → classifies → chunks → embeds → upserts to Qdrant and sets status *ready* (or *failed* + error, audited).

Process — User query → agent routing

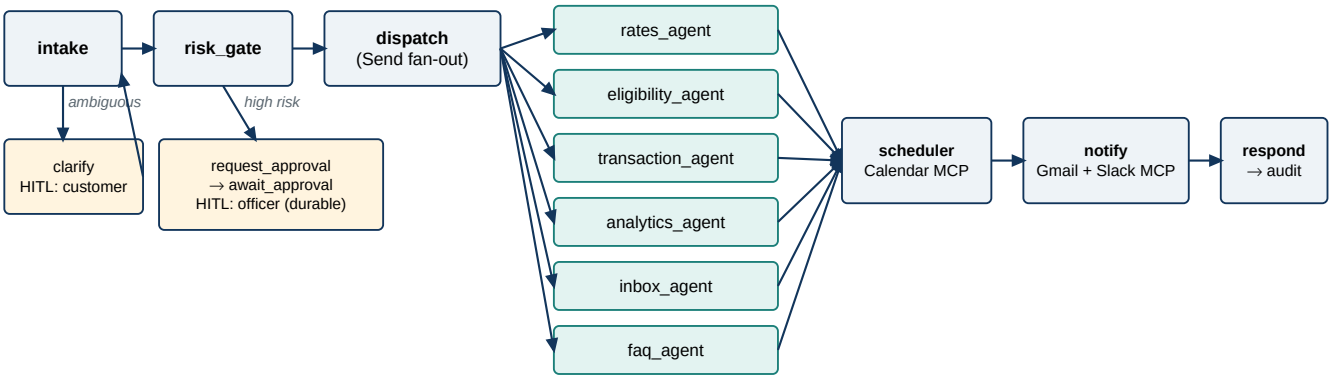
- Streamlit POSTs `/conversations/{id}/messages`; the API takes a Redis lock (one in-flight run per conversation), saves the message and starts the graph on thread `<user_id>:<conversation_id>`. **intake** (LLM structured output → `Understanding`: intents, entities, dates) classifies intent; **risk_gate** applies rules; **dispatch** fans out to the specialists the intents need; results merge into `state.evidence`; **respond** streams the reply back as SSE; **audit** persists the turn. Section 3 details the topology.

2. RAG Pipeline Design (Excel & CSV only)

Stage	Design and reason
Boundary	Enforced at two layers (widget + `validate_upload`) and again implicitly: `load_tables` is the *only* reader and only understands pandas CSV / openpyxl. Nothing else can reach the index.
Parsing	All cells read as strings (no silent type coercion of IDs/dates); headers normalised; empty rows/columns dropped; encoding fallback (utf-8-sig → latin-1); row cap (100k) rejects abusive files. **Every sheet** of a workbook is its own table; each is tagged with file + sheet in every chunk so a 'Home Loan' sheet and a 'Personal Loan' sheet never blur.
Table classification	Heuristic on column names → rates / fees / compliance / eligibility_rules / profile / transactions / other. Used to filter retrieval (the rates agent never reads transaction rows) and to pick the chunk strategy.

Stage	Design and reason
Chunking	Table-aware, three kinds: **schema** chunk per sheet (columns, numeric ranges, categorical values — answers 'what does this file contain'); **row** chunks rendered `[file > sheet] col: value; …` (one row per chunk, grouped ≤40 only for very large tables so a 100k-row file stays within MAX_ROW_CHUNKS); **monthly summary** chunks for transaction tables. Rows are kept intact because a half-row is meaningless.
Embedding	**Hybrid**: dense `BAAI/bge-small-en-v1.5` (384-d, ONNX via fastembed, local) + sparse `Qdrant/bm25`. Dense captures meaning ('mortgage' ≈ 'home loan'); BM25 keeps exact tokens ('750', 'APT-2026…', 'BR001') that dense models blur — essential for tabular data. Local models mean no customer data leaves the host to embed.
Retrieval	Qdrant hybrid query with **RRF fusion**, filtered in the store by `scope=bank OR (scope=customer AND owner_id=caller)`; callers cannot widen it. Retrieved rows are numbered `[S#]` in the prompt; the model must cite tags or say 'not found'. Sources (file, sheet, row range) are shown under each answer.
Exact reads	Numbers are never left to an LLM: profile fields, transaction anomaly flags (large transfer vs the threshold read from the compliance CSV, 3σ outliers, rapid succession, repeated amounts) and all aggregates are computed with pandas (Section 4).

3. Multi-Agent Graph Design (LangGraph)



Conditional edges: intake→clarify|risk_gate · risk_gate→respond|request_approval|specialists · approval→respond|specialists.
Specialists run in parallel; evidence merges into state; clarify and approval pause the thread via interrupt().

- **Nodes:** intake, clarify, risk_gate, request_approval, await_approval, dispatch (router), six specialists (rates, eligibility, transaction, **analytics**, inbox, faq), scheduler, notify, respond, audit. **Conditional edges** implement real branching: ambiguity → clarify loop; out-of-scope → respond; high risk → approval; `dispatch` returns a list of `Send` objects so the needed specialists run **in parallel** (a loan-rate + appointment message runs rates + eligibility concurrently, then scheduler, then notify).
- **State** (`BankState`): identity fields set by the API (user_id, email, tz), `messages` (add_messages reducer), per-turn working memory reset by intake (`evidence` merge-dict so parallel agents write disjoint keys; `actions`/`trace` append-lists), approval and appointment records. Multi-turn memory = the checkpointed thread.
- **Human-in-the-loop:** (1) *customer* — ambiguous requests (e.g. a transfer with no amount) call `interrupt()` in `clarify`; the next chat message resumes the graph. (2) *officer* — transfers ≥ the compliance threshold, fraud reports, large disputes create an `approvals` row, alert #compliance-alerts, and pause durably in `await_approval` (state lives in Postgres, survives restarts). An officer decides in the UI (four-eyes: not their own request); the API enqueues a worker job that resumes the thread and posts the outcome to the customer's conversation. The bot never claims to move money.
- **Failure modes:** every MCP call returns `McpResult(ok, error)` and never raises for expected failures; a Calendar timeout → `appointment.status=failed`, the reply says so, other actions still run; a missing connection is reported as *skipped* with a 'connect it in the sidebar' message (never silent). Scheduling is idempotent (event id derived from the booking reference) and double-booking is prevented by a DB reservation under a per-RM advisory lock plus a Calendar availability re-check inside `create_event`. A Redis lock keeps one run per thread. Dates like 'this Thursday' are resolved in code (`resolve_relative_date`), not by the model.

4. Aggregated Query Handling (process + answers to the assessment's questions)

- **Detection.** Intake classifies `analytics_query` (totals, counts, averages, rankings, trends, comparisons); a regex backstop (aggregate word + a parsed period) catches phrasing small models miss. **Structured compute, not LLM arithmetic:** an LLM *planner* fills a typed `AggPlan` (table, date column, filters, group-by incl. derived month/quarter/year, metrics sum/count/count_distinct/avg/min/max, join, denominator, comparison) for the 4 most relevant tables; `pandas` executes it over **every row** of the full files; the reply text is generated from the numbers, not by the LLM. Metric cards, a table and a chart are rendered from the same result.
- **Dates** come from a deterministic parser (`periods.py`: Q1 2026, April to June, last quarter, first half, last 3 months, financial year with configurable start month) — a wrong date silently changes a banking total, so the model never writes one; model-written date filters are discarded.

- **Multi-file questions.** *Join:* 'branch' lives in Accounts, loans only have account_id → the plan joins on the shared key (left join, right side de-duplicated, noted if keys repeat). *Ratio:* 'recovered as % of total outstanding' = sub-query A / sub-query B over two files (`denominator`, with or without the period). *Comparison:* 'top branch Q1 vs Q4' computes the top group, then the same groups in the previous/second period (delta and %). Multi-sheet workbooks with identical columns also expose a combined `file::*` table.
- **Guardrails against hallucinated aggregates.** (1) Plans are validated against the real schema — unknown table/column ⇒ error fed back for *one* retry, then an explicit 'I couldn't compute that exactly' (no estimate). (2) `repair_plan` fixes predictable model slips from the question text (prose in a numeric filter, counting instead of summing, identifier group-by, missing join) and every repair is shown to the user under 'How this was computed'. (3) Zero matches, a period beyond the data's date coverage, and un-filtered statuses are stated explicitly; empty months are shown as 0 only up to the data's last month. (4) Filter values are parsed strictly (Rs / lakh / crore) — the first live test showed a lenient parser reading prose as 0.1.
- **Accuracy/latency trade-off.** Exactness costs one planner LLM call (+ one retry on invalid plans) and a full-file pandas pass (~tens of ms for 100k rows) instead of a single retrieval; with a local 14B model on CPU planning dominates (~1.5-3 min), with a hosted model seconds. The alternative — retrieving top-K rows and letting the LLM add them up — is faster but wrong by construction, which is unacceptable where an aggregate has compliance consequences. **Verified:** results were compared with independent pandas calculations on the supplied dataset (e.g. Apr–Jun debit 48,190,492.81 / credit 53,495,177.93 month by month; top categories; overdue count) and 30+ unit tests cover each query class.

5. MCP Integration Design

Server	Tools	Why it adds value (not just presence)
Gmail	send_email · list_recent_emails · read_email	Confirmation to the customer, high-priority notice to the RM, summarising a customer's recent mail (content treated as *untrusted data* in prompts).
Calendar	check_availability · find_free_slots · create_event	Availability check before booking; free-slot suggestions on the requested day; invite with branch address, reference number and agenda; conflict ⇒ alternatives, never a double-booking.
Slack	post_message · post_structured · notify_user_by_email	RM DM, #loans-team lead, #compliance-alerts for approvals/high-risk, structured summary of every resolved query to #audit-log. Slack control sequences (<!channel>) are escaped.

- **Tool-selection policy** is explicit, not left to a free-form model: *intents* (extracted by intake) decide which node runs — `schedule_appointment` → scheduler (Calendar); the notify node maps intents/risk to a fixed set (customer email ⇒ Gmail; RM + team ⇒ Slack; high risk ⇒ Slack compliance) and runs them **concurrently** with `asyncio.gather`. This is auditable and testable; free tool-calling would let a prompt-injected email trigger actions.
- **Token injection:** graph → `call_tool(service, state.user_id, tool, args)` → short-lived JWT → MCP server verifies, loads *that* user's token (refreshing if needed) → provider API. Tools accept no user/token arguments. **Failures** (unreachable, not connected, provider error) become `failed` | `skipped` actions with a human-readable reason shown in the reply and the Activity log; every call is audited (service, tool, outcome, latency).

6. OAuth2 Architecture & Token Lifecycle

Two different things. The *OAuth client* (ID + secret) identifies this **application** to Google/Slack and is required to start any consent flow; it grants access to nobody's data. A *user token* is what a person grants when they sign in. The system has exactly one client per provider and **one token per user per service** — there is no service account, no shared mailbox and no token used on behalf of more than one user.

Step	What happens (all server-side)
0 App client	A bank officer opens **Integrations setup**, which lists the exact redirect URIs and scopes to register, and pastes the client ID + secret. It is stored as Fernet ciphertext in `oauth_clients` (its own HKDF context, so it can never be decrypted as a user token), is never returned to the browser (ID masked, secret never shown), can be replaced/removed without a restart (15-second cache so the API and the three MCP processes pick it up) and every change is audited. `GOOGLE_CLIENT_ID/SECRET` and `SLACK_CLIENT_ID/SECRET` in `.env` remain a documented fallback for automated deployments; the database value wins. Google offers no client-less flow and no dynamic client registration, so one registered client is unavoidable.
1 Initiate	User clicks **Connect** in Streamlit → API `POST /oauth/{service}/start` signs a 5-minute *connect ticket* (JWT bound to user + service) → browser opens `mcp-<service>/oauth/start?ticket=…`. The MCP server verifies it, creates a random **state** + PKCE verifier stored in Redis (10 min, single-use, bound to user_id+service) and 302s to the provider (Google: *offline access, prompt=consent, S256 PKCE*; Slack: bot scopes).
2 Authorise	User signs in with **their own** Google/Slack account and grants the scopes below.
3 Callback	Provider → `mcp-<service>/oauth/callback`. The state is *consumed* (`GETDEL`: replay fails), service must match, the code is exchanged (client secret + PKCE verifier), **granted scopes are verified** (Google's granular consent lets users untick permissions → token revoked and rejected), tokens are encrypted and stored under the state's user. Tokens never reach the browser. The MCP server then 302s to Streamlit with `?oauth=<service>&status=ok` plus a 2-minute single-use *resume ticket* that restores the login (the redirect starts a new Streamlit session).

Step	What happens (all server-side)
4 Use & refresh	Each MCP call → `get_access_token(user, service)` under a row lock (`SELECT … FOR UPDATE`, so concurrent calls don't double-refresh): if the Google access token expires within 60 s, the refresh token silently obtains a new one and re-encrypts it. If refresh fails (revoked), the connection is flagged **expired** (Redis flag, audited), tools return `not_connected`, the reply says so and the sidebar card shows ■ **token expired — Reconnect**.
5 Disconnect	✕ in the sidebar → provider revocation endpoint (Google `/revoke`, Slack `auth.revoke`) then the encrypted row is deleted; the card flips to **disconnected** immediately; audited.
Storage	**Fernet** (AES-128-CBC + HMAC-SHA256). The key is derived **per (user, service)** by HKDF-SHA256 from `TOKEN_ENCRYPTION_KEY`, so a blob copied into another user's row cannot be decrypted (unit-tested). Uploaded files use a separate HKDF context.

Service	Exact scopes requested	Justification (minimal for the tools)
Gmail	`openid email` · `gmail.send` · `gmail.readonly`	openid/email: show the connected address. send: confirmation/RM emails. readonly: list/read mail for 'summarise my emails' (no modify/delete).
Google Calendar	`openid email` · `calendar.events` · `calendar.freebusy`	events: create the invite. freebusy: availability without reading event details.
Slack (bot)	`chat:write chat:write.public channels:read users:read users:read.email`	post alerts/DMs; post in channels the bot hasn't joined; resolve channel names; look up the RM by email.

Isolation: a token is only ever loaded with the verified subject of the call's JWT; state binds the flow to the initiating user; tests cover callback state validation, single use, wrong-service state, wrong-user storage, missing-permission rejection, redirect + resume ticket, and per-user encryption.

7. Streamlit Frontend Design

- **Structure:** sidebar = identity + **Integrations & OAuth** cards (connected / disconnected / token expired, account email or workspace, Connect/Reconnect/Disconnect) and preferences; tabs = ■ Chat · ■ Documents · ■ Activity log · ■■■ Approvals · ■ Mock outbox (demo mode only) · ■ Monitoring and ■■ Integrations setup (officers).
- **State:** only the session JWT and user dict live in `st.session_state` (per browser session, never shared); all data is fetched from the API on each rerun. A refresh would otherwise log the user out (Streamlit keeps state per websocket), so the JWT is also kept in a `SameSite=Strict` (Secure on https) cookie that is validated with `/auth/me` on load and deleted on sign-out; Streamlit cannot set HttpOnly cookies, an accepted trade-off noted in Section 11. OAuth redirects land on `/?oauth=…&status=…&resume=…`; `handle_oauth_return` exchanges the resume ticket (`POST /auth/resume`, single-use), shows success/error, and the card fragment refreshes.
- **Chat:** history with clear speaker attribution, live per-agent status ('analytics_agent — computed exactly over 931 rows'), token streaming, agent trace + MCP actions in an expander. **Aggregates render as metric cards + dataframe + bar chart** with a 'How this was computed' expander (periods, plan, repairs); a 'Sources' caption lists file / sheet / rows; 'Clear this conversation' deletes history.
- **Honest status:** a development switch (`MCP_MOCK`) can simulate the three services locally; the cards then read ■ *simulated — no real account connected*, never a green 'connected'. It is off in the submission. **Files:** widget restricted to xlsx/csv, upload progress per file, list with upload time, status, row/chunk counts and error, delete. **Errors:** every API call goes through `ApiError` → friendly message (no stack traces); 409 when a conversation is still processing.

8. Tech Stack Justification

Technology	Role · why chosen over alternatives	Limits / configuration
LLM: pluggable (Ollama qwen2.5:14b local; OpenAI-compatible e.g. Groq; Anthropic)	No vendor lock-in and customer text can stay on-prem (Ollama) for development; hosted models for the demo. LangChain chat-model interface + structured output. Chosen over a single hard-wired API.	CPU inference is slow (minutes). Small models need the repair layer. `LLM_PROVIDER/LLM_MODEL/…`
Embeddings: fastembed bge-small + BM25	Local ONNX, no per-token cost, no data egress; hybrid for exact tokens. Over OpenAI embeddings (egress, cost) and over dense-only (misses '750').	Small model = lower recall on paraphrase; `EMBED_*`
Qdrant	Native dense+sparse hybrid with RRF and payload filters (tenant scoping in the store). Over pgvector (no sparse/RRF) and FAISS (no filters/persistence).	Payloads are plaintext at rest — use disk encryption; single node here
LangGraph 0.4 + Postgres checkpointer	Conditional edges, parallel `Send`, durable `interrupt()` for HITL. Over a plain LangChain chain (no branching/persistence) and CrewAI/AutoGen (less control of state & failure paths).	Checkpoints grow; prune policy not implemented
MCP python SDK (FastMCP, streamable HTTP, stateless)	Standard tool protocol, per-request headers carry the user JWT, scale-out friendly. Over in-process SDK wrappers (no isolation/reuse) and stdio (no per-user headers).	HTTP overhead per call; `MCP_*_URL`
FastAPI (async) + SQLAlchemy/asyncpg, redis.asyncio, httpx	No blocking calls in handlers (bcrypt/pandas run in threads); OpenAPI docs free. Over Flask/Django (sync-first).	—
arq (Redis)	Async-native worker; same Redis as locks/state. Over Celery (heavier, sync-oriented) and RQ (sync).	At-least-once; jobs are idempotent

Technology	Role · why chosen over alternatives	Limits / configuration
Streamlit	Required; fast to build, fragments for polling.	Reruns on every interaction; no native cookies (handled with resume tickets)
OAuth2: hand-written auth-code + PKCE (httpx)	~150 lines, full control to host the flow on the MCP servers, scope verification, revocation, runtime-configurable client. Over Authlib/oauthlib (hide the exact steps the assessment wants explained).	Only Google + Slack providers
Token + client-secret storage: Fernet + HKDF in Postgres	Authenticated encryption; a separate derived key per (user, service) for tokens and a separate context for the app client secret and for uploaded files; no extra service. Over a KMS/Vault (better for production key rotation).	Master key in env; rotating it forces reconnect
Docker Compose	One command, 9 services, healthchecks. Over Kubernetes (overkill for the scope).	Single host

9. Multi-User Architecture

- Each conversation is its own LangGraph thread keyed `user_id:conversation_id`, built server-side from the authenticated JWT; every conversation/document/approval endpoint returns **404** for objects the caller doesn't own. State carries `user_id` from the session, so no node can act for another user.
- A Redis lock allows one in-flight run per thread; asyncio + arq give concurrency across users; `test_concurrent_users_do_not_leak` runs two users in parallel and asserts no cross-over in retrieval scope, MCP calls or answers. Vector queries are filtered by owner in Qdrant; aggregates and profile reads load only the caller's own (+ bank-wide) files.
- OAuth tokens: per-(user, service) rows, per-user derived keys, MCP verifies the JWT subject; contextvars attribute LLM/MCP audit rows to the right user.

10. Banking Domain Considerations

- **Sensitivity:** uploaded files and OAuth tokens encrypted at rest; local embedding/LLM option keeps data on-prem; secrets only in env; tokens never in browser, logs or audit payloads (MCP args logged by key only; LLM prompt/response truncated).
- **No hallucinated numbers:** grounded prompts with citations, exact pandas for aggregates and profiles, explicit 'not found'. **Regulatory:** large-transfer threshold read from the compliance CSV; high-risk requests need officer approval (four-eyes); the assistant never moves money.
- **Compliance logging:** append-only `audit_events` for files, agent steps, MCP calls, LLM calls, OAuth events, approvals, queryable by kind via `/activity` and summarised in the Monitoring tab; an audit summary is mirrored to Slack. **Prompt-injection:** email/Slack/file text is treated as data; tool choice is rule-based; Slack control sequences escaped.

11. Tradeoffs Made Deliberately & What I Would Do With More Time

Deliberate tradeoffs

- **Rule-based tool selection** instead of free LLM tool-calling: less flexible, but auditable and immune to injected instructions.
- **Exact aggregation over embeddings:** one more LLM call and a schema-bound plan language, but correct totals. Not built: free-form SQL generation (a validated plan DSL is safer than executing generated code).
- **Hand-rolled OAuth, Fernet+env master key, single-host Compose**: simple and explainable; no KMS, no HA, no schema migrations (`create_all`), the session cookie is readable by page scripts (no HttpOnly without a reverse proxy in front of Streamlit), no rate limiting beyond login throttling.
- **Data scope:** files are scoped per uploader (customer or bank-wide), not per `customer_id` row: an uploaded multi-customer transaction file is visible to its uploader in full.

With more time

- Map each login to a `customer_id` and enforce row-level security inside the aggregation engine; a KMS-backed key hierarchy with rotation; Alembic migrations; Qdrant payload encryption.
- Evaluation harness with a labelled query set (accuracy of plans/aggregates per model), plan caching for repeated questions, streaming partial tables, a small LLM for planning and a larger one for prose, and OpenTelemetry traces into the monitoring page.
- Token-expiry proactive refresh job, multi-workspace Slack, CI pipeline with a live OAuth contract test against provider sandboxes.